

UT-10: ESTRUCTURAS EXTERNAS DE DATOS

¿Qué vamos a ver?

- Ficheros. Concepto de “persistencia” de los datos de una aplicación.
- Clasificación de ficheros:
 - Contenido: ficheros de texto y ficheros binarios.
 - Modo de acceso: ficheros secuenciales y directos.
- Apertura y cierre de ficheros.
- Lectura y escritura de información en ficheros.
- Almacenamiento de objetos en ficheros. Persistencia. Serialización.

1. FICHEROS. CONCEPTO DE “PERSISTENCIA” DE LOS DATOS DE UNA APLICACIÓN

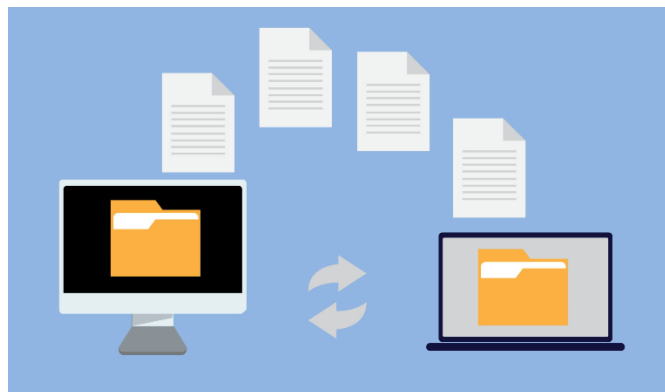


Como ya vimos en la unidad anterior, almacenar la información en memoria, como hacíamos antes de conocer los ficheros XML, tiene el problema de que al finalizar el programa toda esa información se pierde.

Sin embargo, gracias los ficheros podremos almacenar la información que necesitemos, de forma permanente, con la posibilidad de recuperarla más adelante. Al concepto de almacenar la información para luego poder recuperarla de nuevo se le denomina “persistencia de datos”, y podremos aplicarla tanto al contexto de ficheros como al de bases de datos.

Un **fichero** es una colección de datos organizados que se almacenan en un soporte externo de almacenamiento, como un disco duro, disco compacto, etc. Estos soportes no se borran cuando finaliza la ejecución del programa, ni siquiera cuando se apaga el ordenador.

Los ficheros no sólo nos serán útiles a la hora de almacenar la información, sino que también será muy útiles a la hora de transferir información de un dispositivo a otro.



2. CLASIFICACIÓN DE FICHEROS

Es posible realizar distintas clasificaciones de ficheros en función de determinadas características. En nuestro caso nos vamos a centrar en las clasificaciones por el tipo de contenido y por el modo de acceso.

2.1. Contenido

Principalmente podemos distinguir dos tipos de ficheros en función de su contenido:

2.1.1. Ficheros de texto

Los datos se guardan en los ficheros en forma de texto, es decir, como secuencias de caracteres. El contenido de este tipo de ficheros puede ser leído por una persona.

2.1.2. Ficheros binarios

Los datos son guardados en su representación binaria, es decir, en secuencias de bytes. El contenido de este tipo de ficheros no puede ser leído por una persona.

2.2. Modo de acceso

Según el modo de acceso a los datos, los ficheros se pueden clasificar en:

2.2.1. Ficheros secuenciales

Se trata de archivos en los que el contenido se lee o escribe de forma continua. No se accede a un punto concreto del archivo. Para leer cualquier información necesitamos leer todos los datos desde el principio del fichero hasta llegar a la información deseada. En general los archivos de texto se suelen utilizar de forma secuencial.

2.2.2. Fichero de acceso directo

Se puede acceder a cualquier dato del archivo conociendo la posición en el mismo. Dicha posición se suele indicar en bytes.

3. APERTURA Y CIERRE DE FICHEROS

Para manejar ficheros vamos a hacer uso de la clase `java.io.File`.

3.1. Creación de ficheros

Para crear un fichero usaremos el constructor de la clase `File`, proporcionándole el nombre del fichero que queremos crear, y después ejecutaremos el método `createNewFile()`.

```
public File(String pathname)
```

```
public boolean createNewFile() throws IOException
```

Como podemos observar, el método `createNewFile` propaga excepciones `IOException`, por tanto, cuando trabajemos con ficheros en Java, siempre vamos a necesitar encapsularlo dentro de un `try...catch`

Ejemplo:

```
File archivo = new File("archivo.txt");  
archivo.createNewFile();
```

Esto nos creará el fichero “archivo.txt” en la misma ruta que se encuentra el proyecto.

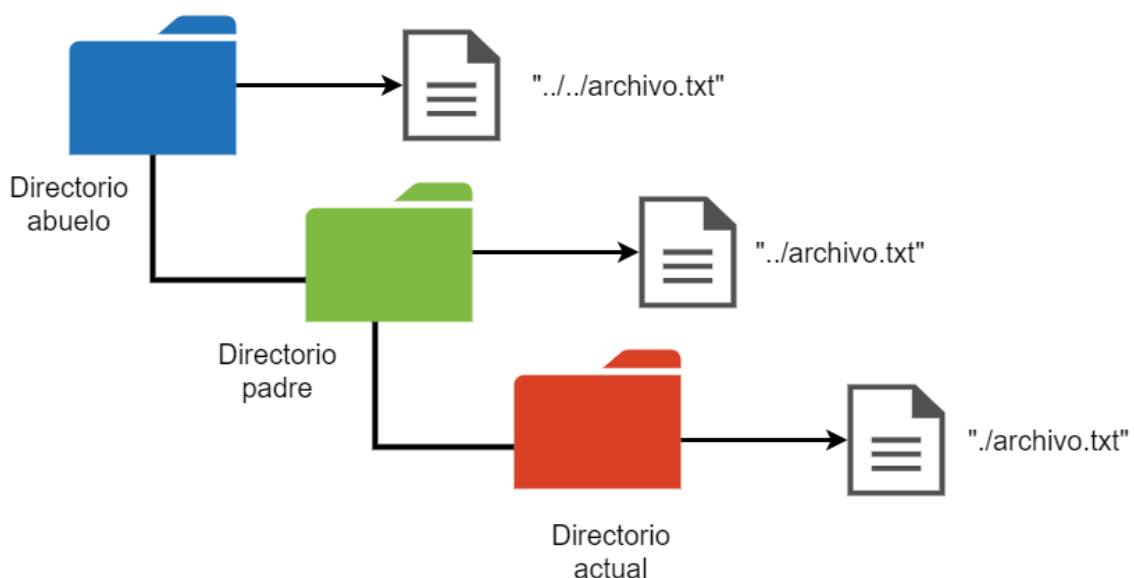
En el caso de queramos crear el fichero en una ruta específica, debemos dar la o la ruta absoluta o la ruta relativa del mismo. Por supuesto, la ruta de directorios deberá existir, de lo contrario nos dará error.

Ejemplo con ruta relativa:

```
File archivo01 = new File("./ruta01/archivo01.txt");  
archivo01.createNewFile();
```

Cuando utilizemos rutas relativas debemos tener en cuenta varias cosas:

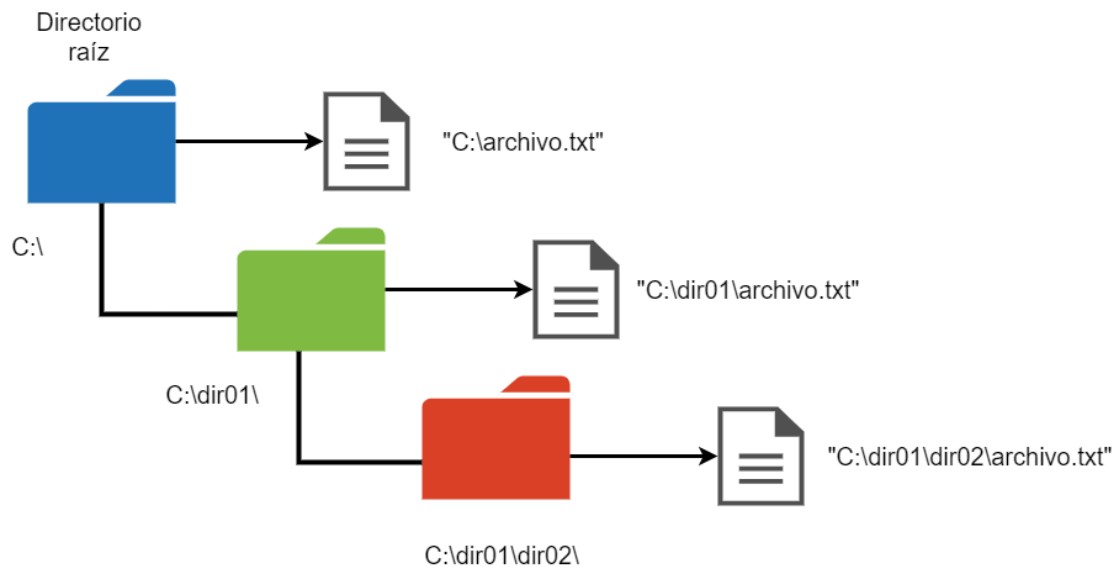
- En las rutas relativas se toma como referencia el directorio actual, es decir, el del proyecto.
- Si utilizamos “./” al comienzo de la ruta estamos indicando que comience desde el directorio actual.
- Si utilizamos “../” al comienzo de la ruta estamos indicando que comience desde el directorio padre del directorio actual. Podremos unir varios “../” para subir niveles de directorios, por ejemplo, con “../..” nos posicionaremos en directorio padre del directorio padre del directorio actual.
- Para separar niveles de directorios utilizamos el carácter “/”, por ejemplo, “../..ruta03/archivo.txt”



Ejemplo con ruta absoluta:

```
File archivo02 = new File("c:\\ruta02\\archivo02.txt");
archivo02.createNewFile();
```

Cuando utilizemos rutas absolutas debemos escribir toda la ruta completa, desde el directorio raíz. Para dividir los niveles de directorios debemos utilizar "\\".



El código completo quedaría de la siguiente manera:

```
public static void main(String[] args) {
    try {
        File archivo01 = new File("./ruta01/archivo01.txt");
        if(archivo01.createNewFile())
            System.out.println("El fichero se creó");
        else
            System.out.println("El fichero no se creó");
    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
```

3.2. Creación de directorios

Para crear directorios utilizaremos los métodos `mkdir()` y `mkdirs()`.

```
public boolean mkdir()
```

```
public boolean mkdirs()
```

La diferencia entre `mkdir` y `mkdirs` consiste en que `mkdirs` creará también los directorios padre si no existen, mientras que `mkdir` solo crea el directorio final y si los directorios padres no existen no lo creará.

A la hora de crear los directorios debemos tener en cuenta los mismos factores que con la creación de ficheros, ya sean con ruta relativa o con ruta absoluta.

Veamos un ejemplo:

```
public static void main(String[] args) {
    try {
        File directorio01 = new File("./directorio01");
        if(directorio01.mkdir())
            System.out.println("Se ha creado el directorio01");
        else
            System.out.println("No se ha creado el directorio01");

        File directorio02 = new
File("C:\\\\directorioPadre\\\\directorio02");
        if(directorio02.mkdirs())
            System.out.println("Se ha creado el directorio02");
        else
            System.out.println("No se ha creado el directorio02");

    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
```

3.3. Eliminación de ficheros y directorios

Si lo que necesitamos es eliminar ficheros directorios haremos uso del método "delete()".

```
public boolean delete()
```

Previamente crearemos un objeto de la clase File con un archivo o directorio existente y después ejecutaremos delete().

Cuando eliminemos un directorio, también eliminaremos todo su contenido.

```
public static void main(String[] args) {  
    try {  
  
        File archivo02 = new File("c:\\ruta02\\archivo02.txt");  
        if(archivo02.delete())  
            System.out.println("archivo02 fue eliminado");  
        else  
            System.out.println("archivo02 fue eliminado");  
  
        File ruta01 = new File("./ruta01/");  
        if(ruta01.delete())  
            System.out.println("El directorio rut01 fue eliminado");  
        else  
            System.out.println("El directorio rut01 fue eliminado");  
  
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
}
```

3.4. Renombrar o mover ficheros y directorios

Para renombrar o mover ficheros y directorios vamos a utilizar el mismo método:

```
public boolean renameTo(File dest)
```

En el parámetro introduciremos un objeto File con la nueva ruta y/o nombre.

```
public static void main(String[] args) {
    try {

        File archivo01 = new File("./ruta01/archivo01.txt");
        File archivo01renamed = new
File("./ruta01/archivo01renamed.txt");
        if(archivo01.renameTo(archivo01renamed))
            System.out.println("El fichero se renombró");
        else
            System.out.println("El fichero no se renombró");

        File archivo02 = new File("./ruta01/archivo02.txt");
        archivo02.createNewFile();
        File archivo02moved = new File("./archivo02.txt");
        if(archivo02.renameTo(archivo02moved))
            System.out.println("El fichero se movió");
        else
            System.out.println("El fichero se movió");

        File ruta01 = new File("./ruta01/");
        File directorio01 = new File("./directorio01/ruta01/");
        if(ruta01.renameTo(directorio01))
            System.out.println("El directorio fue movido");
        else
            System.out.println("El directorio fue movido");

    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
```

3.5. Leer contenido de directorios

Para conocer el contenido de un directorio tenemos dos opciones.

En primer lugar, podemos obtener los objetos File correspondientes a todos los archivos y directorios que se encuentran en el propio directorio con el método:

```
public File[] listFiles()
```

Este método nos devuelve un array de objetos File.

Si lo que nos interesa son solo los nombres, podemos utilizar:

```
public String[] list()
```

Veámoslo en código:

```
public static void main(String[] args) {  
    try {  
  
        File directorio = new File("./nbproject/");  
        File[] archivos = directorio.listFiles();  
        for(File archivo:archivos){  
            System.out.println(archivo.getName());  
        }  
  
        System.out.println("");  
  
        String[] archivosString = directorio.list();  
        for(String archString: archivosString){  
            System.out.println(archString);  
        }  
  
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
}
```

Link de la ayuda oficial de java de la clase File:

<https://docs.oracle.com/javase/8/docs/api/java/io/File.html>

4. LECTURA Y ESCRITURA DE INFORMACIÓN EN FICHEROS



4.1. Lectura de información binaria en ficheros

Para leer el contenido de un fichero en formato binario, es decir, byte a byte, utilizaremos la clase `java.io.FileInputStream`, clase hija de `java.io.InputStream` que ya vimos en la unidad 2 con la entrada/salida estándar.

Para crear un objeto de lectura de fichero por bytes tendremos que pasar como parámetro la ruta del fichero que deseamos leer o un objeto `File` que haga referencia a dicho fichero.

```
public FileInputStream(File file) throws FileNotFoundException
```

```
public FileInputStream(String name) throws FileNotFoundException
```

```
FileInputStream lector1 = new FileInputStream("./archivo02.txt");  
File archivo03 = new File("./archivo03.txt");  
FileInputStream lector2 = new FileInputStream(archivo03);
```

Para leer los datos de este objeto tenemos dos opciones principales, leer carácter a carácter o en bloques de `n` caracteres.

```
public int read() throws IOException
```

```
public int read(byte[] b) throws IOException
```

Hay que destacar que la primera opción nos devuelve un valor entero, correspondiente al byte que ha leído, siendo -1 en el caso de no leer nada, mientras que a la segunda opción pasamos por parámetro un array de bytes que se rellenará, total o parcialmente, con el contenido leído, devolviendo el número de bytes que ha leído. El resultado será -1 si no ha leído nada.

Veamos un ejemplo de cada uno:

```
int palabra;
do{
    palabra = lector2.read();
    if (palabra == -1)
        break;
    else{
        System.out.print(palabra + " ");
    }
}while(true);
Lector2.close();
```

```
byte[] lectura = new byte[100];
int longitud;

do{
    longitud = lector1.read(lectura);
    if (longitud == -1)
        break;
    else{
        for(int i=0; i<longitud; i++){
            System.out.print(lectura[i] + " ");
        }
    }
    System.out.println("");
}while(true);
lector1.close();
```

Algo muy importante, que no debemos olvidar, es cerrar el fichero después de terminar de leer y/o escribir en él, ya que, de lo contrario, puede quedarse bloqueado y no podremos acceder a su contenido.

Para ello vamos a utilizar el método `close()`.

4.2. Lectura de información en ficheros de texto

Para este tipo de lectura contamos con dos opciones.

4.4.1. FileReader y BufferedReader

En primer lugar tenemos la clase `java.io.FileReader`, la cual podemos combinar con `java.io.BufferedReader` para leer línea a línea en lugar de carácter a carácter.

Para crear un objeto `FileReader` podemos pasar por parámetro un objeto `File` o el nombre del fichero que queremos leer.

```
public FileReader(File file) throws FileNotFoundException
```

```
public FileReader(String fileName) throws FileNotFoundException
```

Y a la hora de crear el objeto `BufferedReader` pasaremos por parámetro el objeto `FileReader` que hayamos creado previamente.

```
public BufferedReader(Reader in)
```

Ejemplos:

```
File archivo = new File("archivo.txt");  
FileReader lector1 = new FileReader(archivo);  
BufferedReader lectorBuff1 = new BufferedReader(lector1);
```

```
FileReader lector2 = new FileReader("archivo.txt");  
BufferedReader lectorBuff2 = new BufferedReader(lector2);
```

El objeto `FileReader`, de por sí, ya nos permitirá leer el contenido del fichero, pero principalmente carácter a carácter, mediante el método `read()`, que devuelve un entero correspondiente al valor del carácter.

Mientras que si utilizamos el objeto `BufferedReader` podremos realizar también la lectura carácter a carácter, con `read()`, o línea a línea con `readLine()`. Este último método nos devolverá el `String` correspondiente a la línea que ha leído, con un resultado de `null` en el caso de llegar al final del fichero.

Lectura carácter a carácter:

```
int dato;
do{
    dato = lector1.read();
    if(dato == -1)
        break;
    else
        System.out.print(dato + " ");
}while(true);
lector1.close();
System.out.println("");
```

Lectura línea a línea:

```
String linea;
do{
    linea = lectorBuff2.readLine();
    if(linea == null)
        break;
    else
        System.out.println(linea);
}while(true);
lectorBuff2.close();
System.out.println("");
```

4.4.1. Scanner

La segunda opción para realizar la lectura del contenido de un fichero de texto será utilizando la clase Scanner pasándole un objeto File por parámetro, con el cual indicaremos de qué fichero queremos leer.

En este caso solo podremos utilizar los métodos `next()` y `nextLine()` para recoger línea a línea los datos del fichero. También haremos uso de los métodos `hasNext()` y `hasNextLine()` para realizar una comprobación previa de si queda alguna línea por leer.

```
String datos;

File ficheroDatos = new File("archivo03.txt");
Scanner lectorDatos = new Scanner(ficheroDatos);
while (lectorDatos.hasNextLine()) {
    datos = lectorDatos.nextLine();
    System.out.println(datos);
}

lectorDatos.close();
```

Links a la ayuda de java de todas las clases que hemos visto para la lectura de ficheros:

<https://docs.oracle.com/javase/8/docs/api/java/io/FileInputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/FileReader.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

<https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html>

4.3. Escritura de información binaria en ficheros

En el caso de la escritura binaria de ficheros haremos uso de la clase `java.io.FileOutputStream`, clase hija de `java.io.OutputStream`.

A la hora de crear el objeto de la clase `FileOutputStream` contamos con varias opciones.

Por un lado, podemos elegir, como ocurría con `FileInputStream`, entre pasar como parámetro un objeto `File` o directamente la ruta del fichero que queremos utilizar.

```
public FileOutputStream(File file) throws FileNotFoundException
```

```
public FileOutputStream(String name) throws FileNotFoundException
```

Con estos dos constructores, en el caso de que el fichero ya existe, se eliminará todo su contenido previamente a realizar la escritura. Es decir, que perderemos todo el contenido que tenía antes de escribir sobre él.

En el caso de que lo que necesitemos sea añadir contenido, tenemos dos constructores, con los que podemos indicar esta opción.

```
public FileOutputStream(File file, boolean append) throws  
FileNotFoundException
```

```
public FileOutputStream(String name, boolean append) throws  
FileNotFoundException
```

Estos dos constructores incluyen un parámetro boolean, denominado append, de tal forma que, si le pasamos true, lo que escribamos se añadirá al final del fichero, pero si pasamos false, se eliminará el contenido actual y se escribirá sobre el fichero vacío.

Para escribir en el fichero, utilizaremos el método write, al cual podremos pasarle como parámetro un array de bytes o un entero correspondiente al valor de un byte.

```
byte[] array = new byte[26];  
  
FileOutputStream escritor1 = new FileOutputStream ("archivo02.txt");  
for(int i = 0; i < 26; i++){  
    escritor1.write(i);  
    array[i] = (byte) i;  
}  
escritor1.close();  
  
FileOutputStream escritor2 = new FileOutputStream ("archivo02.txt",  
true);  
escritor2.write(array);  
escritor2.close();
```

4.4. Escritura de información en ficheros de texto

Al igual que ocurría con la lectura de ficheros de texto, en la escritura tenemos dos opciones.

4.4.1. *FileOutputStream, OutputStreamWriter y BufferedWriter*

La primera es utilizando `java.io.FileOutputStream`, combinado con `java.io.OutputStreamWriter` y `java.io.BufferedWriter`.

El procedimiento consiste en crear un objeto `FileOutputStream`, utilizar este objeto para crear un objeto `OutputStreamWriter`, y después utilizar este último para crear un `BufferedWriter`.

```
public OutputStreamWriter(OutputStream out)
```

```
public BufferedWriter(Writer out)
```

El método `write()` será el encargado de la propia escritura en el fichero. A este es método podremos pasarle por parámetro tanto un carácter, como una cadena de caracteres.

```
FileOutputStream fos = new FileOutputStream("archivo01.txt");
OutputStreamWriter osw = new OutputStreamWriter(fos);
BufferedWriter fw = new BufferedWriter(osw);

String[] cadenas = {"Hola,", "¿qué", "tal?"};

for(String cadena:cadenas){
    fw.write(cadena + " ");
}
fw.write("\n");
for(String cadena:cadenas){
    fw.write(cadena + "\n");
}
fw.close();
osw.close();
fos.close();
```

4.4.2. *FileWriter y PrintWriter*

La segunda opción es utilizar la clase `java.io.PrintWriter` en combinación con `java.io.FileWriter`.

FileWriter cuenta con cuatro constructores que nos permiten pasar como parámetro un objeto File, un String con la ruta del fichero, y también nos da la opción de indicar si queremos eliminar el contenido del fichero, si ya existe, o si por el contrario deseamos añadir la nueva información al final del mismo.

```
public FileWriter(File file) throws IOException
```

```
public FileWriter(File file, boolean append) throws IOException
```

```
public FileWriter(String fileName) throws IOException
```

```
public FileWriter(String fileName, boolean append) throws IOException
```

Al constructor de PrintWriter le vamos a pasar el objeto FileWriter que obtengamos.

```
public PrintWriter(Writer out)
```

Como métodos de escritura, PrintWriter cuenta con una gran cantidad, entre los que destacan:

- **print:** que nos permitirá escribir en el fichero datos de tipo primitivo como caracteres, enteros, números en coma flotante, ... y también strings.
- **println:** que funciona igual que “print”, pero este escribirá un salto de línea al final.
- **write:** con él podemos escribir caracteres, arrays de caracteres y Strings.

```
FileWriter archivo = new FileWriter("archivo0.txt", false);
PrintWriter escritor = new PrintWriter(archivo);
String[] cadenas = {"Hola,", "¿qué", "tal?"};

for(String cadena:cadenas){
    escritor.print(cadena + " ");
}
escritor.println("");
for(String cadena:cadenas){
    escritor.println(cadena + " ");
}
```

```
}  
escriptor.close();
```

A continuación, tenéis los links a la ayuda de java de todas las clases que hemos visto para la escritura:

<https://docs.oracle.com/javase/8/docs/api/java/io/FileOutputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/OutputStreamWriter.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedWriter.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/FileWriter.html>

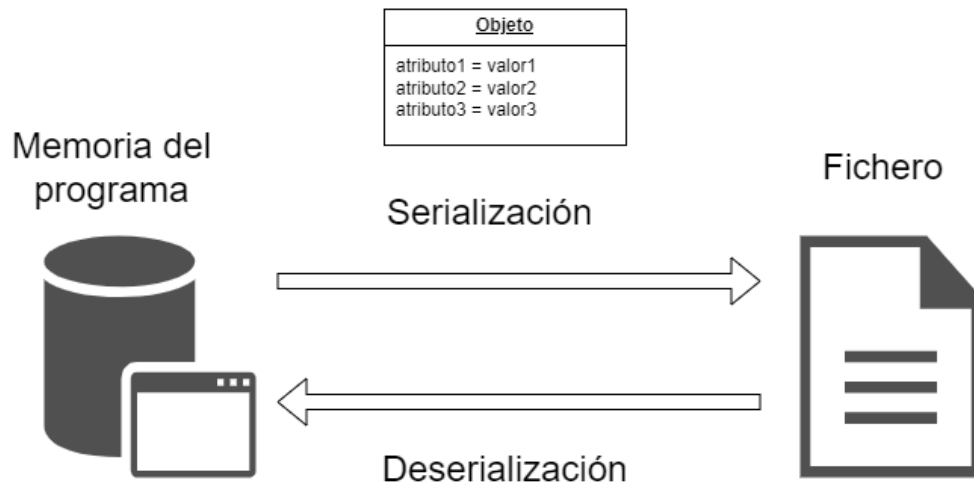
<https://docs.oracle.com/javase/8/docs/api/java/io/PrintWriter.html>

5. ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA. SERIALIZACIÓN.

Además de datos binarios, o texto, java nos permite persistir, es decir, almacenar y recuperar, objetos en ficheros.

Esto consistirá en coger objetos que tengamos almacenados en memoria y los podamos introducir en los ficheros en formato binario, de tal forma que cuando recuperemos esos datos podremos guardarlos de nuevo en objetos del mismo tipo en memoria.

A la operación de almacenar objetos en ficheros le damos el nombre de serialización, mientras que la operación inversa, es decir, recoger los datos del fichero y guardarlo en un objeto le denominamos deserialización.



5.1. Serialización

A la hora de almacenar objetos en ficheros vamos a hacer uso de las clases `java.io.FileOutputStream` y `java.io.ObjectOutputStream`. La primera será la que realice la escritura en el fichero, mientras que la segunda se encargará de transformar el objeto a una serie de bytes que se puedan escribir en el fichero.

En primer lugar crearemos un objeto `FileOutputStream`, y luego pasaremos dicho objeto al constructor de `ObjectOutputStream`.

```
protected ObjectOutputStream() throws IOException, SecurityException
```

Después almacenaremos el objeto a través del método `writeObject`, pasando como parámetro el objeto en cuestión.

```
Objeto objeto = new Objeto("Nombre1", 1, "Descripción1");
FileOutputStream fos = new FileOutputStream("objetos.dat");
ObjectOutputStream oos = new ObjectOutputStream(fos);

oos.writeObject(objeto);
oos.close();
fos.close();
```

Es importante tener en cuenta que solo podremos serializar objetos de clases que implementen la interfaz `Serializable`. En el caso de nuestras propias clases, debemos hacer que nuestra clase implemente dicha interfaz.

```
public class Objeto implements Serializable {  
    private String nombre;  
    private Integer id;  
    private String descripcion;  
  
    ...  
}
```

5.2. Deserialización

Para recuperar los objetos de un fichero utilizaremos las clases `java.io.FileInputStream` y `ObjectInputStream`.

En esta ocasión crearemos un objeto `FileInputStream` y luego se lo pasaremos como atributo al constructor de `ObjectInputStream`.

```
public ObjectInputStream(InputStream in) throws IOException
```

Para extraer los objetos haremos uso del método `readObject`, el cual tendremos que utilizar con un casteo explícito para almacenar la información en un objeto de su clase.

```
Objeto objeto2;  
FileInputStream fis = new FileInputStream("objetos.dat");  
ObjectInputStream ois = new ObjectInputStream(fis);  
  
objeto2 = (Objeto) ois.readObject();  
ois.close();  
System.out.println(objeto2.toString());
```

En el caso de necesitar leer todos los objetos de un fichero, debemos comprobar que en el `FileInputStream` aún quedan caracteres por leer, antes de realizar cada lectura.

Veamos un ejemplo:

```
FileInputStream fis = null;
ObjectInputStream ois = null;
Persona persona;
try {
    fis = new FileInputStream(ruta);
    ois = new ObjectInputStream(fis);

    do{
        if(fis.available()==0)
            break;

        persona =(Persona) ois.readObject();
        System.out.println(persona.toString());
    } while (true);
} catch (Exception e) {
    e.toString();
}
```

Links a la ayuda de java:

<https://docs.oracle.com/javase/8/docs/api/java/io/ObjectOutputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/ObjectInputStream.html>